

`nil` and `void`

Expressing absence in data literal syntax and beyond.

What is the difference between these?

```
(update-user-info "alice"  
                  { :name "Alice Doe" })
```

```
(update-user-info "alice"  
                  { :name      "Alice Doe"  
                    :address nil })
```

What is the difference between these?

:: Update the name **only.**

```
(update-user-info "alice"  
                  { :name "Alice Doe" })
```

:: Update the name **and clear the address.**

```
(update-user-info "alice"  
                  { :name      "Alice Doe"  
                    :address nil })
```


Programmatically constructing that input

```
(defn build-update-user-info [changes]
  (-> {}
    (assoc-some :name (:name changes))
    (assoc-some :address (build-address changes))
    (assoc-some :email (:email-punycode changes))))
```

```
(defn build-address [{:keys [address]}]
  (-> {}
    (assoc-some :postcode (fmt-zip (:zip address)))
    (assoc-some :country (:country address))))
```

Can we make the values that our functions produce, more obvious?

**Can we represent this optionality
in the literal syntax?**


```
{:foo 1  
  :bar nil}
```

=>

```
{:foo 1  
  :bar nil}
```

#voidable

{:foo 1
 :bar **void**}

=>

{:foo 1}

#voidable

{:foo 1

 :bar **#voidable**

 {:baz **void**}}

=>

{:foo 1

 :bar {}}}

#voidable

{:foo 1

 :bar **#supervoidable**

 {:baz **void**}}

=>

{:foo 1}

```
(defn build-update-user-info [changes]
  (-> {}
    (assoc-some :name (:name changes))
    (assoc-some :address (build-address changes))
    (assoc-some :email (:email-punycode changes))))
```

```
(defn build-address [{:keys [address]}]
  (-> {}
    (assoc-some :postcode (fmt-zip (:zip address)))
    (assoc-some :country (:country address))))
```



```
(defn build-update-user-info [changes]
  #voidable
  {:name      (:name changes void)
   :email     (:email-punycode changes void)
   :address #supervoidable
            {:post-code (fmt-zip (:zip changes void))
             :country  (:country changes void)}})
```

You can use it too

But that's just a proof of concept...

- The Clojure prototype works using macros but is limited to data literal syntax.
 - Maps: `{}`
 - Sets: `#{}`
 - Lists: `()` <- making it useful for writing macros.
 - Vectors: `[]`
- <https://codeberg.org/axvr/archive/src/branch/master/2026/void>

Can we take it further?

Deep integration of a **void** value into the language.

- Semantics of a **void** value:
 - Superset of `nil`. Should behave exactly like `nil` except:
 - Self-destructive when encountered by a collection type.
 - If a data structure contains no value at index or key, return **void**, not `nil`.
 - A function that returns nothing should return **void**?
- Alternatively a value (maybe `nil`) could be *tagged* as "voided"?
 - `^:void nil`

Have I seen something like this before?

Comparing **void** to other alternatives.

- Alternatives are generally distinct from `null/nil` and behave subtly differently or are more difficult to use.
 - JavaScript example: a list of `[1, undefined, 3]` contains 3 elements.
- Capable of superseding the functionality of any alternative while maybe being more useful, particularly if doing functional programming.
- There are work arounds for mimicking this behaviour (one of which I showed before) which may be good enough for your use cases.

Problems of a **void** value

- Deep integration is hard:
 - Subclassing `nil` is not possible in many languages.
 - Built-in collections may not be able to be extended to handle **void**.
- Debugging in some cases may become more difficult:
 - Accidental voiding: where did that data go?
 - Would need good schema validation of inputs and outputs.
 - The benefits *may* outweigh the negatives. Further investigation is needed.

Find out more

Come speak to me afterwards too.

- Try it out: <https://codeberg.org/axvr/archive/src/branch/master/2026/void>
- Me: <https://www.alexvear.com> | alex@vear.uk